Software Test Plan (STP)

Project: Healthcare Appointment and Patient Record Manager

Version: 1.0 (initial draft)

Authors:

KARANAM SREEHITHA – PES1UG24AM129

KHUSHI PATEL – PES1UG24AM134

KIRTHANA S -- PES1UG24AM136

PORLA NEHA -- PES1UG25AM808

Status: Reviewed by Project Team

Revision History

Version	Date	Author	Change Summary	Approval
1.0	28-10-2026	Project team	Initial project-specific draft adapted from provided template	Approved by Project Team

Approvals

Role	Name	Signature / Email	Date
Team Lead	KARANAM SREEHITHA	sreehithakaranam@gamil.com	28-10-2026
Team Member	KHUSHI HIRENKUMAR PATEL KIRTHANA S PORLA NEHA	khushihpatel06@gmail.com Kirthanashetty17@gmail.com porlaneha1@gmail.com	28-10-2026
Course TA / Evaluator			

1. Introduction

Purpose: define the scope, approach, environment, schedule, responsibilities and evidence for testing the Healthcare Appointment and Patient Record Manager.

Scope: current verified patient and appointment workflows, validation, database persistence, health endpoint and error paths. Authentication, medical history, prescription management and dashboard testing are conditional on those features being implemented.

References: project SRS, SAD, source code, pytest test suite and repository CI workflow.

Definitions: SRS = Software Requirements Specification; SAD = Software Architecture and Design; STP = Software Test Plan; RTM = Requirements Traceability Matrix; UAT = User Acceptance Testing.

2. Test Items

- Patient registration, listing/search, details, editing and deletion.
- Appointment booking, listing/filtering, editing and status updates.
- Input validation and controlled error handling.
- SQLite persistence and patient/appointment relationships.
- Health endpoint.
- Automated tests and GitHub Actions CI checks.
- Planned items when implemented: authentication/authorization, medical history, prescription management and dashboard.

3. Features to be Tested

Test ID	Test scenario	Requirement(s)	Expected result
TC-PAT-01	Register patient with valid data	FR-001	Patient record is stored and displayed.
TC-PAT-02	Submit patient form with missing/invalid fields	FR-002	Validation rejects invalid data; no invalid record is stored.
TC-PAT-03	Search for existing and non-existing patient	FR-004	Matching records appear; no-match result is handled clearly.
TC-PAT-04	View patient details	FR-005	Correct patient's details are shown.
TC-PAT-05	Edit patient with valid and invalid data	FR-006	Valid update persists; invalid update is rejected.
TC-PAT-06	Delete patient and inspect related appointment behavior	FR-007	Deletion follows defined relationship constraints and produces controlled feedback.
TC-APT-01	Book appointment with valid data	FR-008	Appointment is stored and visible.
TC-APT-02	Book appointment with invalid data	FR-009	Validation rejects invalid values.
TC-APT-03	List and filter appointments	FR-010, FR-011	Results match the selected filters.
TC-APT-04	Edit appointment	FR-012	Valid changes persist.
TC-APT-05	Update appointment status	FR-013	Supported status change persists.
TC-APP-01	Call health endpoint	FR-014	Expected health response is returned.
TC-ERR-01	Request missing patient/appointment ID	FR-015	Controlled response; no unhandled exception.
TC-NFR-01	Measure representative local response time	NFR-001	Record timing under documented test conditions.
TC-NFR-02	Create/update/retrieve record	NFR-002	Stored values match submitted valid values.
TC-NFR-03	Inspect logs and error responses	NFR-006, SR-005	No secrets or real patient data are exposed.

4. Features Not to be Tested

- External hospital, insurance, payment or third-party integrations that are not part of the verified application.
- Production-scale load, high availability and disaster recovery unless a deployment environment and measurable targets are defined.
- Clinical correctness of medical advice or prescription decisions; the project is an administrative coursework prototype.
- Hardware integrations; no dedicated hardware interface is identified.
- Planned modules that have not been implemented. These must be added to scope when ready.

5. Test Approach / Strategy

5.1 Levels

- Unit/component testing: validate patient and appointment validation helpers and isolated behavior where testable.
- Integration testing: exercise Flask routes with the test client and a temporary/test SQLite database.
- System testing: execute end-to-end patient and appointment workflows through the browser or Flask test client.
- Acceptance testing: have a teammate or reviewer complete representative workflows against the SRS acceptance criteria.

5.2 Types

- Functional testing of patient and appointment workflows.
- Negative and boundary-value testing for missing, malformed and invalid inputs.
- Regression testing after code changes.
- Data-integrity testing for persistence, edits, deletion and relationships.
- Basic usability checks for labels, navigation, validation and empty states.
- Security-oriented checks for input handling, secret/data exposure and access control if implemented.

Entry criteria: dependencies installed; database/test fixtures can initialize; a stable branch/build is available; test data is synthetic.

Exit criteria: all planned tests have been executed; failures are fixed or documented; no unresolved critical defect blocks the demonstrated core workflow; CI is green; requirement status and known gaps are documented.

5.3 Security Validation

- Test server-side validation with missing, malformed and boundary-value inputs.
- Check that error pages and logs do not reveal stack traces, credentials or unnecessary patient details.
- Verify that no real patient data or secrets are present in tracked files.
- Test authentication/authorization only after the corresponding feature is implemented; do not claim a security pass before that.

6. Test Environment

Item	Environment / notes
OS	Developer workstation; current team development has used Windows.
Runtime	Python 3.11-compatible environment for this project; confirm requirements.txt before a clean install.
Application	Flask application from the project repository.
Database	SQLite; tests should use an isolated temporary/test database rather than production/local user data.
Test tools	pytest; coverage tooling when configured; browser for manual UI checks.
CI	GitHub Actions workflow to install dependencies and run tests on pull requests/pushes; verify workflow file and run before claiming configured.
Test data	Synthetic patients and appointments only.

7. Test Schedule

Use the TA's latest schedule as the controlling schedule. Sprint 1: 27 Oct–31 Oct 2026. Sprint 2: 2 Nov–7 Nov 2026. Presentation readiness begins 9 Nov 2026. The three documents are due before Sprint 1 begins according to the latest instruction. Adjust actual execution dates to the team's Jira/GitHub Project plan.

Milestone	Target	Evidence
Finalize SRS, SAD and STP	Before Sprint 1	Reviewed documents in the team repository.
Sprint 1 implementation and tests	27–31 Oct 2026	Updated backlog, passing tests and 1-minute demo video if required by TA.
Sprint 2 implementation and regression tests	2–7 Nov 2026	Updated backlog, test results and 2-minute demo video if required by TA.
Presentation readiness	From 9 Nov 2026	Working product, documented deviations and final review.

8. Test Deliverables

- This Software Test Plan.
- Automated test cases and fixtures in tests/.
- Test execution output and coverage report where configured.
- Defect/issue records in the team tracker.
- CI run evidence and demo video(s) required by the TA.
- Final test summary with pass/fail counts, known limitations and requirement coverage.

9. Roles and Responsibilities

Role	Name	Responsibility
Team Lead / coordinator	KARANAM SREEHITHA	Coordinate deliverables, review requirements/traceability and monitor repository/CI status.
Developer / tester	KHUSHI PATEL	Assigned implementation and test responsibilities to be confirmed in the project board.
Developer / tester	KIRTHANA S	Assigned implementation and test responsibilities to be confirmed in the project board.
Developer / tester	PORLA NEHA	Assigned implementation and test responsibilities to be confirmed in the project board.
TA / evaluator	To be assigned	Review deliverables and evaluate demonstrations.

10. Risks and Mitigation

Risk	Mitigation
Late or unstable changes	Keep changes small, use pull requests and run regression tests.
Test data contaminates developer database	Use isolated test fixtures and synthetic data.
Feature scope unclear	Track implemented/planned status in SRS and review with the team.
CI pipeline fails on clean environment	Pin/document dependencies and reproduce the workflow locally.
Documentation diverges from code	Update RTM and documents after significant sprint changes.

11. Assumptions & Dependencies

- Project dependencies are installable from the committed requirements file.
- The test suite can create and use an isolated database.
- All test records are synthetic.
- The TA's latest dates take precedence over older dates in the supplementary classroom text, as directed by the team.
- Exact story assignments and story points are maintained in the team project board, not invented in this document.

12. Suspension & Resumption Criteria

Suspend a test run if the app cannot start, the test database is unavailable, or a blocking defect prevents execution of a substantial part of the planned suite. Resume after the blocking defect is fixed, the environment is restored and a smoke test passes.

13. Test Case Management & Traceability

Use stable test IDs and map each test to SRS requirement IDs. Record execution status as Not Run, Pass, Fail or Blocked. Update the RTM when requirements or test cases change.

Requirement	Test case(s)	Current evidence/status
FR-001 to FR-007	TC-PAT-01 to TC-PAT-06	Patient tests reported passing in latest run; verify individual mappings before final sign-off.
FR-008 to FR-013	TC-APT-01 to TC-APT-05	Appointment tests reported passing in latest run; verify individual mappings before final sign-off.
FR-014 to FR-015	TC-APP-01, TC-ERR-01	Covered by tests as reported; confirm exact case mapping.
NFR / SR requirements	TC-NFR-01 to TC-NFR-03 and future security tests	Some require manual measurement or future implementation; do not mark passed without evidence.

14. Test Metrics & Reporting

- Number of test cases planned, executed, passed, failed and blocked.
- Pass percentage = passed / executed × 100 (report the denominator).
- Requirement coverage: requirements with at least one executed test / requirements in scope.
- Defects by severity and age.
- Code coverage, when measured, reported separately from requirement coverage.

Latest reported baseline from the team: 31 automated tests passed and 85% app.py coverage. Treat these as the latest reported run, not a guarantee of future runs; rerun the suite from the final commit and attach fresh evidence.Team

Team :

Team member	USN
KARANAM SREEHITHA	PES1UG24AM129
KHUSHI PATEL	PES1UG24AM134
KIRTHANA S	PES1UG24AM136
PORLA NEHA	PES1UG25AM808